



上海交通大学

SHANGHAI JIAO TONG UNIVERSITY

基于 CATArena 平台的双层 Chess Agent 系统 项目建议书

组员 1 夏子壹 524031910002

组员 2 韩雨阳 524031910095

组员 3 陈芊羽 524031910093

摘要

本项目面向 CATArena 多轮竞技评测场景，拟构建一个由 Chess Playing Agent 与 Optimization Agent 组成的双层 Chess Agent 系统，并研究其在多轮反馈条件下的持续优化能力。与传统棋类程序仅追求单局对弈强度不同，本项目关注代码型 Agent 是否能够在标准化接口、受限执行环境与对手持续变化的外部条件下，利用比赛结果、运行日志、对手策略与排行榜信息完成稳定迭代。技术路线上，项目以 Alpha-Beta 剪枝搜索为基础决策框架，引入迭代加深、走法排序、静态搜索与置换表等增强机制，并以 Stockfish/NNUE 评估体系作为基准支撑；同时结合 ReAct、Reflexion 与 Self-Refine 等方法所体现的推理执行协同、自我反思和反馈驱动优化思想，构建双层 Agent 的信息闭环。项目将依托 CATArena 的 tournament 机制设计可复现的多轮实验，从胜率、排名变化、非法行棋率、超时率、版本自我对弈增益与反馈利用有效性等维度评估系统性能，进而验证双层 Agent 架构在复杂策略任务中的工程可行性与研究价值。

目录

1 项目背景与研究意义	4
2 国内外研究现状与技术基础	4
2.1 Agent 研究现状	4
2.2 代码型 Agent 与自我优化 Agent	4
2.3 国际象棋 AI 发展现状	4
2.4 Stockfish 与 NNUE 评估体系	5
2.5 CATArena 平台及其评测价值	5
3 研究目标与拟解决的关键问题	5
3.1 研究目标	5
3.2 拟解决的关键问题	5
4 系统总体方案与关键技术路线	6
4.1 总体架构	6
4.2 推理与决策机制	6
4.3 反馈驱动优化机制	6
4.4 状态管理与历史信息利用机制	6
4.5 双层 Agent 协同流程	7
5 国际象棋决策与优化算法设计	8
5.1 搜索算法	8
5.1.1 基础方案: Alpha-Beta 剪枝搜索	8
5.1.2 增强机制: 迭代加深、走法排序、静态搜索与置换表	8
5.1.3 增强方案: 蒙特卡洛树搜索	8
5.2 局面评估算法	8
5.2.1 Centipawn 评估	8
5.2.2 WDL 评估	8
5.2.3 Mate Distance 评估	8
5.2.4 NNUE 评估网络	8
5.3 学习与优化机制	9
5.3.1 Baseline 构建	9
5.3.2 模仿学习方案	9
5.3.3 反馈驱动策略改进	9
6 实验平台、数据来源与评测方案	9
6.1 实验平台与数据来源	9
6.2 多轮 tournament 数据组织方式	9
6.3 评测方案	9
6.3.1 双模块架构的预期能力	10
6.3.2 Chess Playing Agent 的执行可靠性评估	10
6.3.3 Optimization Agent 的演化有效性评估	10
6.3.4 Agent 执行过程的行为可审计性评估	11

7 研究计划与预期成果	12
7.1 阶段性研究安排	12
7.2 预期成果	12
8 参考文献	12

1 项目背景与研究意义

大语言模型驱动的智能体系统正在从单轮问答任务扩展到需要长期规划、工具调用、环境交互与自我修正的复杂任务环境。在此背景下，如何评估 Agent 是否具备持续优化能力，已成为智能体研究中的关键问题之一。仅以单次任务成败衡量系统性能，难以揭示其在真实软件开发与复杂决策问题中的长期行为质量；相比之下，多轮反馈、版本演化和竞争性评测更能反映 Agent 的稳定性、适应性与改进能力。

国际象棋具有规则明确、状态空间复杂、反馈信号清晰和评价体系成熟等特点，是研究 Agent 决策能力与反馈利用能力的理想载体。一方面，棋类环境能够为搜索算法、评估函数和策略优化机制提供标准化试验场；另一方面，棋局胜负、非法走子、超时、局面崩溃和对手针对性克制等现象，又为分析 Agent 的失效机理提供了高质量反馈。

CATArena 为代码型 Agent 提供了统一接口、多轮对抗、版本更新和排行榜机制，使研究对象从“能否写出一个可运行程序”进一步转向“能否在竞争环境中持续改进程序”。基于此，本项目拟构建双层 Chess Agent 系统：底层 Chess Playing Agent 负责单局对弈决策，上层 Optimization Agent 负责基于多轮反馈实施策略修订与代码优化。项目的研究意义在于：以国际象棋这一高结构化任务为载体，系统考察代码型 Agent 在多轮反馈条件下的自我改进机制，进而为复杂软件任务中的自动优化 Agent 提供可迁移的方法论参考。

2 国内外研究现状与技术基础

2.1 Agent 研究现状

近年来，面向复杂任务的大语言模型 Agent 研究强调“推理、行动与反馈”的闭环组织。ReAct 框架指出，将显式推理与外部行动交替结合，有助于提升模型在交互式任务中的鲁棒性与可解释性 [1]。在评测层面，AgentBench 等工作进一步表明，Agent 的能力不仅体现在语言生成质量上，更体现在环境适应、工具使用和长期决策等综合维度 [4]。相关研究为本项目提供了重要启发，即在棋类环境中不应将 Agent 视为静态文本生成器，而应将其建模为能够感知状态、调用算法模块并依据结果迭代修正的动态系统。

2.2 代码型 Agent 与自我优化 Agent

代码型 Agent 的核心任务已从“一次性生成程序”逐步扩展为“生成、执行、诊断与修订”的持续闭环。Self-Refine 证明了语言模型可通过自反馈机制对既有输出进行多轮修正 [3]；Reflexion 将文字化反思引入强化式迭代过程，强调失败经验在后续决策中的作用 [2]；SWE-agent 则将这一思路推进到软件工程任务中，显示出基于工具链和执行反馈的自动修复潜力 [5]。这些工作共同表明，面向复杂代码任务的有效 Agent 往往需要显式建模错误来源、记录历史决策并对修订结果实施验证。本项目的双层 Agent 架构正是在这一研究脉络下提出：Playing Agent 负责局内决策，Optimization Agent 负责跨轮次诊断与优化，从而实现代码型 Agent 的持续演化。

2.3 国际象棋 AI 发展现状

国际象棋人工智能长期沿着“搜索算法 + 局面评估”两条主线演进。传统强引擎以 Minimax/Alpha-Beta 框架为核心，通过高效剪枝与启发式排序在有限时间内搜索更深层博弈树 [6, 7, 8]。现代开源引擎在此基础上进一步引入迭代加深、置换表、静态搜索等机制，以提升搜索稳定性与战术敏感性。

与此同时, AlphaZero 证明了神经网络与蒙特卡洛树搜索结合的自博弈框架能够在棋类任务中取得极高水平 [10]。这说明国际象棋决策既可依赖确定性搜索与高效评估, 也可借助统计搜索和学习机制提升全局规划能力。对本项目而言, 国际象棋 AI 的成熟技术栈为 Playing Agent 的设计提供了可靠基线, 也为 Optimization Agent 提供了明确的优化目标。

2.4 Stockfish 与 NNUE 评估体系

Stockfish 是国际象棋领域最具代表性的开源引擎之一, 其优势在于将高性能搜索框架与工程化评估体系紧密结合 [11]。自引入 NNUE 评估网络后, Stockfish 在保持高搜索效率的同时显著提升了局面评估质量。NNUE 的关键特征在于可增量更新, 即当局面发生局部变化时, 仅对相关特征进行快速重算, 从而适配搜索树中大量相邻节点的评估需求 [12]。对本项目而言, Stockfish/NNUE 一方面可以作为强基线与外部评测器, 另一方面也可作为 Playing Agent 的候选走法排序、局面诊断与版本效果验证提供统一标尺。

2.5 CATArena 平台及其评测价值

CATArena 面向代码型 Agent 的持续演化问题构建了多轮竞技评测框架, 平台强调在统一约束下比较不同 Agent 的初始能力与演化能力 [14]。与单轮静态评测相比, CATArena 能够提供每轮比赛结果、排行榜、执行日志、历史版本与对手信息等结构化反馈, 使研究者能够观察 Agent 是否真正基于反馈完成定向修正。对本项目而言, CATArena 不只是部署环境, 更是验证双层架构有效性的实验基础: 若系统能够在该平台上实现稳定接入、合法对弈、有效迭代与排名提升, 则说明其具备面向真实约束环境的持续优化潜力。

3 研究目标与拟解决的关键问题

3.1 研究目标

本项目拟在 CATArena 的 chessgame 环境中构建双层 Chess Agent 系统, 并围绕以下目标开展研究:

1. 构建具备稳定对弈能力的 Chess Playing Agent, 使其能够在规范时间约束内输出合法、有效且具有基本竞争力的走法。
2. 构建面向多轮 tournament 反馈的 Optimization Agent, 使其能够基于比赛结果、日志与对手信息实施定向修订。
3. 建立可复现的版本管理、性能比较和反馈利用分析流程, 验证系统是否具备持续迭代优化能力。
4. 形成一套适用于代码型棋类 Agent 的实验评价方法, 为后续扩展到其他策略任务提供参考。

3.2 拟解决的关键问题

围绕上述目标, 项目重点解决以下问题:

1. 在受限执行环境中, 如何将规则解析、搜索、评估与安全降级机制整合为稳定的 Playing Agent。
2. 在多轮反馈条件下, 如何区分局部失误、系统性缺陷与对手针对性克制, 并将其转化为有效的优化指令。

3. 如何避免 Optimization Agent 产生仅停留于文本层面的“伪改进”，确保其修改能够在后续比赛中体现为可观测性能增益。
4. 如何建立兼顾棋力、稳定性、演化趋势与合规性的综合评价指标体系，以支撑系统性比较分析。

4 系统总体方案与关键技术路线

4.1 总体架构

系统采用双层结构。Chess Playing Agent 面向局内决策，负责接收棋盘状态、生成合法走法集合、执行搜索与评估并输出最终行动。Optimization Agent 面向轮间优化，负责在每轮比赛结束后读取结构化反馈，识别关键失效模式，形成版本修订方案并生成下一轮候选实现。两层之间通过“对弈表现 - 反馈诊断 - 版本修订 - 再部署”的闭环形成协同。

与单一 Agent 架构相比，双层设计的优势在于职责分离。Playing Agent 侧重局部最优决策与运行稳定性，Optimization Agent 侧重跨轮次模式分析与策略调整，从而避免将即时决策与长期优化混合在同一控制循环中导致的目标冲突。

4.2 推理与决策机制

Playing Agent 的控制逻辑吸收了 ReAct 的方法启发，但不保留显式的提示词轨迹式表达。具体而言，系统在每次行动时按照“状态解析 - 候选生成 - 搜索评估 - 结果校验 - 行动输出”的顺序组织决策。该机制的关键并非展示自然语言推理痕迹，而是确保模型或程序化模块能够在每一步调用外部搜索器、评估器或规则校验器时获得一致的中间信息表示。若候选走法存在非法输出、评估异常或时间预算超限，系统将触发安全降级逻辑，从当前合法走法集合中返回保底动作，以保证比赛流程不因局部错误而中断。

4.3 反馈驱动优化机制

Optimization Agent 的设计参考 Reflexion 与 Self-Refine 所体现的“执行结果驱动修订”思想。每轮 tournament 结束后，系统按照“反馈聚合 - 失效诊断 - 对手分析 - 修订规划 - 版本验证”的路径完成迭代。其核心不是机械复写高排名对手代码，而是在统一评估口径下识别影响性能的关键因素，例如搜索深度设置不合理、走法排序弱、残局处理不足、开局选择单一、非法走子保护缺失或时间控制过于激进等。随后，系统将这些诊断结论映射为可执行的代码或参数修改，并通过自测或回放验证其是否真正改善行为。

4.4 状态管理与历史信息利用机制

为支持跨轮次分析与局部回溯，项目采用结构化状态管理机制。对于 Playing Agent，每局对弈保留 FEN、走法序列、评估分数、时间消耗与关键错误标记等记录，用于赛后定位失效局面。对于 Optimization Agent，则保留版本号、修改摘要、对应反馈证据、验证结果与后续轮次表现，以形成可追踪的演化链条。该机制可防止优化过程退化为不可解释的随机改写，同时为分析“哪类修改真正提升了性能”提供证据基础。

4.5 双层 Agent 协同流程

双层协同遵循四阶段闭环：第一，Playing Agent 进入比赛环境并完成多局对弈；第二，平台返回比赛结果、日志、对手信息和排行榜；第三，Optimization Agent 汇总反馈并生成修订方案；第四，新版本重新部署至下一轮 tournament。该流程的实质是将局内决策能力与局间演化能力同时纳入评估框架，使系统能够在统一平台上接受连续检验。

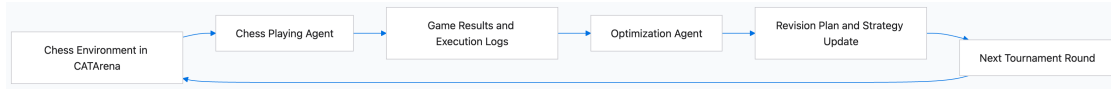


图 1: 双层 Agent 总体架构图

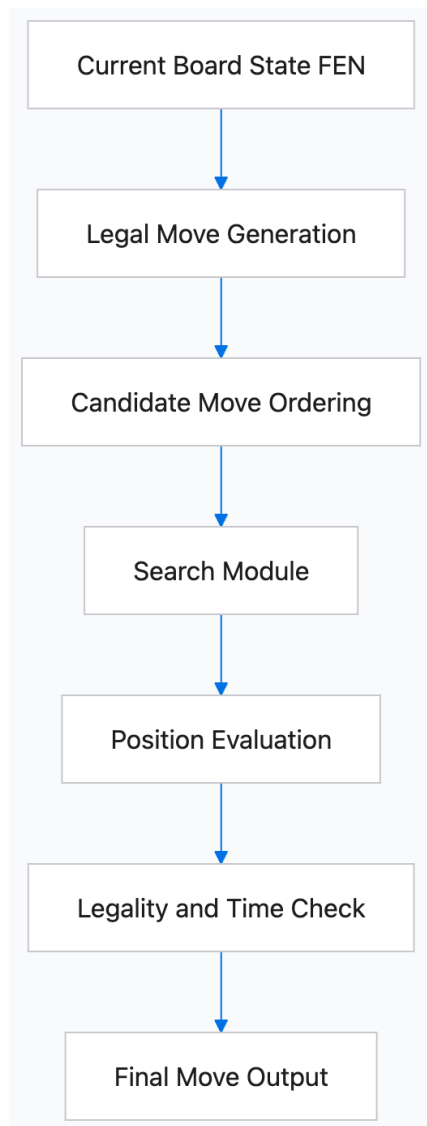


图 2: Playing Agent 决策流程图

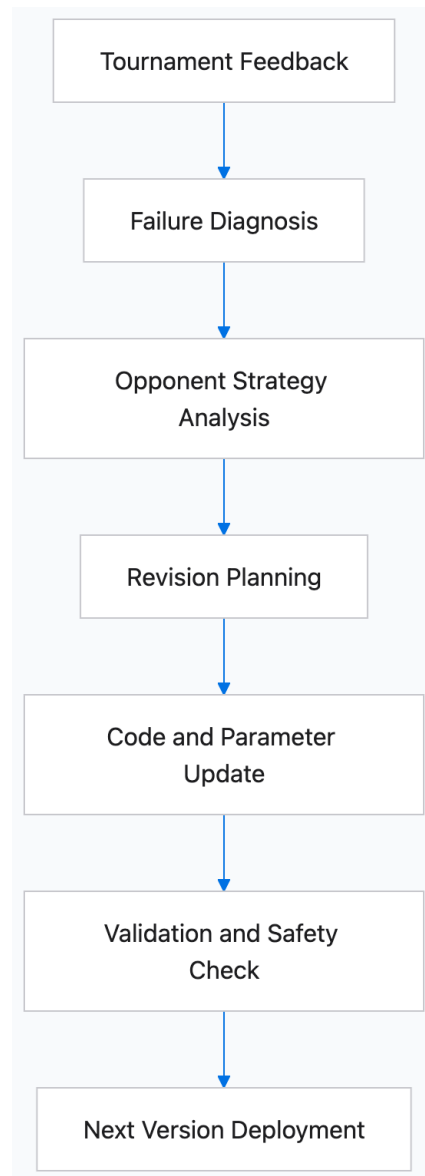


图 3: Optimization Agent 迭代优化流程图

5 国际象棋决策与优化算法设计

5.1 搜索算法

5.1.1 基础方案：Alpha-Beta 剪枝搜索

Alpha-Beta 剪枝是本项目 Playing Agent 的基础搜索框架。其原因在于该方法与国际象棋的零和、完全信息、轮流行动特性高度契合，能够在确定时间预算内提供稳定、可解释且工程实现成熟的决策能力 [6, 7]。相较于直接依赖纯语言推断，基于搜索的决策更能保证基本棋力下界与行为一致性。

5.1.2 增强机制：迭代加深、走法排序、静态搜索与置换表

为提升基础搜索性能，项目在 Alpha-Beta 框架上引入四类增强机制。其一，迭代加深用于在时间受限情形下优先获得浅层可用解，并以较浅层最佳走法指导更深层搜索；其二，走法排序通过优先考察吃子、将军、升变和历史最佳走法，提高剪枝效率；其三，静态搜索用于处理战术不稳定局面，减少地平线效应；其四，置换表用于缓存重复局面的评估结果，降低重复计算开销。这些方法共同构成 Playing Agent 的基础工程能力，也是后续 Optimization Agent 优先优化的核心对象。

5.1.3 增强方案：蒙特卡洛树搜索

蒙特卡洛树搜索作为增强方案而非初始默认方案引入 [9]。其优势在于对复杂局面和长期收益具有更强的统计估计能力，但其计算成本、参数敏感性与实现复杂度相对更高。基于本科项目的时间与资源边界，本项目优先以 Alpha-Beta 系统建立稳定基线，再在特定复杂局面、时间较宽松的对局条件或扩展实验中评估 MCTS 与确定性搜索的混合潜力。

5.2 局面评估算法

5.2.1 Centipawn 评估

厘兵值是最基础的量化评估指标，用于统一表达局面优劣程度。其优势在于可直接服务于搜索中的节点比较、阈值剪枝与版本性能对照，因此适合作为 Playing Agent 的基础内部评分尺度。

5.2.2 WDL 评估

胜平负概率评估适用于将数值优势转换为更贴近比赛结果的统计量。相较于单纯厘兵值，WDL 指标更适合用于跨版本比较和轮间分析，尤其是在优势较大但 cp 值不再线性反映胜率时，可为 Optimization Agent 提供更稳定的性能解释依据。

5.2.3 Mate Distance 评估

在存在强制杀棋序列时，杀棋距离应高于一般数值评估优先级。将其纳入评估体系，有助于避免 Agent 在明显强制战术局面中被一般位置分数误导，从而提升残局和战术收束阶段的决策质量。

5.2.4 NNUE 评估网络

NNUE 作为现代强引擎的关键组件，兼具高效率与较强评估能力，适合作为本项目的基准评估器与外部分析器。项目初期并不以自主训练大型评估网络为目标，而是将 NNUE 主要用于候选走法

评价、关键败局诊断与版本对照分析，以控制系统复杂度并保证研究重心聚焦于双层 Agent 的演化机制。

5.3 学习与优化机制

5.3.1 Baseline 构建

项目首先构建基线方案，包括规则正确性、接口适配、合法走子保障和基本搜索能力。必要时可设置以 Stockfish 建议走法或轻量启发式策略为参照的外部基线，用于判定 Playing Agent 的最低可用水平。

5.3.2 模仿学习方案

若项目进度允许，可利用高质量局面数据与强引擎分析结果构建监督信号，以“局面 - 候选走法 - 评分”为样本训练轻量策略模型或排序模型。该方案的作用主要在于提升候选走法排序质量，而非完全替代搜索框架。

5.3.3 反馈驱动策略改进

本项目的核心学习机制并非大规模离线训练，而是基于 tournament 反馈的增量式优化。Optimization Agent 将根据错误类型和对手表现，优先修改搜索参数、启发式排序规则、时间控制、开局偏好与异常保护逻辑。这一方案更符合 CATArena 对代码型 Agent 持续演化能力的评测目标，也是本项目区别于单纯棋类引擎复现工作的关键所在。

6 实验平台、数据来源与评测方案

6.1 实验平台与数据来源

项目以 CATarena 的 chessgame 环境作为核心实验平台，并基于 python-chess 完成棋盘状态解析、合法走法生成与对局记录管理 [13]。平台每轮可提供以下几类数据：比赛结果与得分、对局日志与回放、非法行棋与异常信息、对手提交版本、排行榜变化以及自身历史版本记录。上述数据共同构成双层 Agent 的反馈基础。

6.2 多轮 tournament 数据组织方式

为保证可复现性，项目将以“轮次 - 版本 - 对局 - 反馈”四层结构组织数据。每一轮数据记录至少包含：版本标识、核心修改摘要、对局结果统计、关键失败局面、异常事件、对手特征与下一轮修订方案。该组织方式既支持按轮追踪性能变化，也支持按失效类型检索案例，为后续归因分析提供基础。

6.3 评测方案

本项目针对 CATarena 框架下的 Chess 任务设计了双模块 Agent 架构。Chess Playing Agent 负责单轮棋局执行，Optimization Agent 负责轮间策略演化。评估目标聚焦于三个方面：执行可靠性、演化有效性与行为可审计性。

6.3.1 双模块架构的预期能力

Chess Playing Agent 应具备稳定参赛能力，能够正确解析 CATArena 提供的棋局状态，在规定时间内输出合法棋步，并在异常场景下保持鲁棒性，避免接口错误、非法动作或运行崩溃影响比赛流程。

Optimization Agent 应具备轮间优化能力，能够读取比赛反馈、排行榜、执行日志和对手策略，分析自身策略短板与对手优势，并将反馈信息转化为可解释的代码改进，推动策略性能随轮次持续提升。

在全部过程中，还需要保持安全、合规和可审计。

6.3.2 Chess Playing Agent 的执行可靠性评估

针对 Chess Playing Agent，重点评估其在单轮比赛中的接口合规性、动作合法性与运行稳定性。评估内容包括动作是否始终属于当前合法走法集合，是否在规定时间内完成响应，接口输出格式是否符合 CATArena 要求，以及是否因逻辑错误或运行异常导致默认动作触发。

设 Agent 共行动 T 次，第 t 次输出动作为 a_t ，当前合法动作集合为 $\mathcal{A}_t$ ，则合法动作率定义为：

$$R_{\text{legal}} = \frac{1}{T} \sum_{t=1}^T \mathbb{I}(a_t \in \mathcal{A}_t).$$

设成功返回符合接口规范输出的次数为 T_{success} ，则接口成功率定义为：

$$R_{\text{interface}} = \frac{T_{\text{success}}}{T}.$$

设运行错误次数为 T_{error} ，则运行错误率定义为：

$$R_{\text{error}} = \frac{T_{\text{error}}}{T}.$$

设第 t 次行动的响应时间为 τ_t ，每步最大允许时间为 τ_{max} ，则平均响应时间与超时率分别定义为：

$$\bar{\tau} = \frac{1}{T} \sum_{t=1}^T \tau_t,$$

$$R_{\text{timeout}} = \frac{1}{T} \sum_{t=1}^T \mathbb{I}(\tau_t > \tau_{\text{max}}).$$

上述指标共同衡量 Playing Agent 是否能够作为稳定、可靠的比赛执行模块接入 CATArena。

6.3.3 Optimization Agent 的演化有效性评估

针对 Optimization Agent，采用 CATArena 原论文定义的 S_{base} 与 S_{evo} 作为核心指标。当前 Chess 任务中仅有两个参赛 Agent，因此公式可在原始定义基础上进行简化。

设第 i 个 Agent 在第 n 轮提交的策略为 $C_i^{(n)}$ ，其对手记为 j 。策略 $C_i^{(n)}$ 对阵 $C_j^{(m)}$ 的平均得分记为：

$$W_{i,j}^{n,m}.$$

对于 Chess 任务，单局得分规则为：

$$s = \begin{cases} 1, & \text{win,} \\ 0.5, & \text{draw,} \\ 0, & \text{lose.} \end{cases}$$

若两份策略之间重复对弈 K 局，则平均得分为：

$$W_{i,j}^{n,m} = \frac{1}{K} \sum_{k=1}^K s_{i,j,k}^{n,m}.$$

基础能力分数 S_{base} 衡量 Agent 在第一轮尚未经过反馈优化时的初始策略水平。

$$S_{\text{base}}(i) = W_{i,j}^{1,1}.$$

全局轮次表现 $G_i^{(n)}$ 用于衡量第 i 个 Agent 在第 n 轮策略的整体强度。按照 CATArena 原始定义， $G_i^{(n)}$ 是该轮策略对阵所有 Agent、所有轮次策略的平均表现。

$$G_i^{(n)} = \frac{1}{2N-1} \sum_{\substack{m=1 \\ (j,m) \neq (i,n)}}^N W_{i,j}^{n,m}.$$

在本项目的简化实验设置中，若仅进行同轮对局评估，也可采用同轮表现：

$$\tilde{G}_i^{(n)} = W_{i,j}^{n,n}.$$

演化能力分数 S_{evo} 衡量 Agent 表现随轮次推进的增长趋势。根据 CATArena 原文方法，对 $G_i^{(n)}$ 关于轮次 n 进行普通最小二乘线性回归：

$$G_i^{(n)} = \alpha_i + S_{\text{evo}}(i)n + \epsilon_i^{(n)}.$$

其中， $S_{\text{evo}}(i)$ 为回归斜率，其最小二乘估计为：

$$S_{\text{evo}}(i) = \frac{\sum_{n=1}^N (n - \bar{n})(G_i^{(n)} - \bar{G}_i)}{\sum_{n=1}^N (n - \bar{n})^2},$$

其中：

$$\bar{n} = \frac{1}{N} \sum_{n=1}^N n, \quad \bar{G}_i = \frac{1}{N} \sum_{n=1}^N G_i^{(n)}.$$

当 $S_{\text{evo}}(i) > 0$ 时，说明 Agent 的策略性能随轮次呈正向提升趋势。当 $S_{\text{evo}}(i) < 0$ 时，说明轮间优化可能导致策略退化。

6.3.4 Agent 执行过程的行为可审计性评估

除功能与演化能力外，Agent 在真实执行过程中的行为也需要接受审计。评估重点包括是否完整读取 CATArena 返回的关键反馈文件，例如 tournament report、history、执行日志和对手代码，是否在授权范围内调用工具和访问文件，是否能够将代码修改追溯到明确的反馈依据，是否存在越权访问、环境篡改、非法复制对手代码或规避评测逻辑等安全风险行为。上述内容可通过日志回溯、工具调用记录和代码变更记录进行验证。

7 研究计划与预期成果

7.1 阶段性研究安排

阶段	主要任务
第一阶段	完成文献调研，明确双层架构、平台接口与评测指标，搭建基础实验环境。
第二阶段	实现 Playing Agent 基线版本，完成规则解析、合法走子、基础搜索与异常保护。
第三阶段	实现 Optimization Agent，建立反馈读取、失败诊断、版本管理与修订验证流程。
第四阶段	开展多轮 tournament 实验，对搜索增强、评估策略与修订方案进行对比分析。
第五阶段	整理实验结果，撰写结题报告与研究论文初稿，完善演示材料。

7.2 预期成果

本项目预期形成以下成果：

1. 一个可在 CATArena 平台稳定运行的双层 Chess Agent 原型系统。
2. 一套覆盖功能稳定性、棋力表现与演化能力的实验评价方案。
3. 若干轮可复现实验结果，包括版本演化记录、关键失败案例与性能变化分析。
4. 一份较完整的项目研究报告，并争取形成课程论文或学术竞赛支撑材料。

8 参考文献

参考文献

- [1] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[EB/OL]. arXiv:2210.03629, 2022.
- [2] Shinn N, Cassano F, Berman E, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[EB/OL]. Advances in Neural Information Processing Systems, 2023.
- [3] Madaan A, Tandon N, Gupta P, et al. Self-Refine: Iterative Refinement with Self-Feedback[EB/OL]. Advances in Neural Information Processing Systems, 2023.
- [4] Liu X, Yu B, Feng Y, et al. AgentBench: Evaluating LLMs as Agents[EB/OL]. arXiv:2308.03688, 2023.
- [5] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[EB/OL]. arXiv:2405.15793, 2024.
- [6] Newell A, Shaw J C, Simon H A. Chess Playing Programs and the Problem of Complexity[J]. IBM Journal of Research and Development, 1958, 2(4): 320-335.

- [7] Knuth D E, Moore R W. An Analysis of Alpha-Beta Pruning[J]. Artificial Intelligence, 1975, 6(4): 293-326.
- [8] Russell S, Norvig P. Artificial Intelligence: A Modern Approach[M]. 4th ed. Hoboken: Pearson, 2021.
- [9] Browne C B, Powley E, Whitehouse D, et al. A Survey of Monte Carlo Tree Search Methods[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2012, 4(1): 1-43.
- [10] Silver D, Hubert T, Schrittwieser J, et al. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play[J]. Science, 2018, 362(6419): 1140-1144.
- [11] Stockfish Developers. Stockfish Chess Engine[EB/OL]. <https://github.com/official-stockfish/Stockfish>.
- [12] Chessprogramming Wiki. NNUE[EB/OL]. <https://www.chessprogramming.org/NNUE>.
- [13] Fiekas N. python-chess Documentation[EB/OL]. <https://python-chess.readthedocs.io/>.
- [14] Tian Y, Chen Q, Zhang C, et al. CATArena: An Autonomous Evolution Arena for Code Agents[EB/OL]. arXiv:2510.26852, 2025.